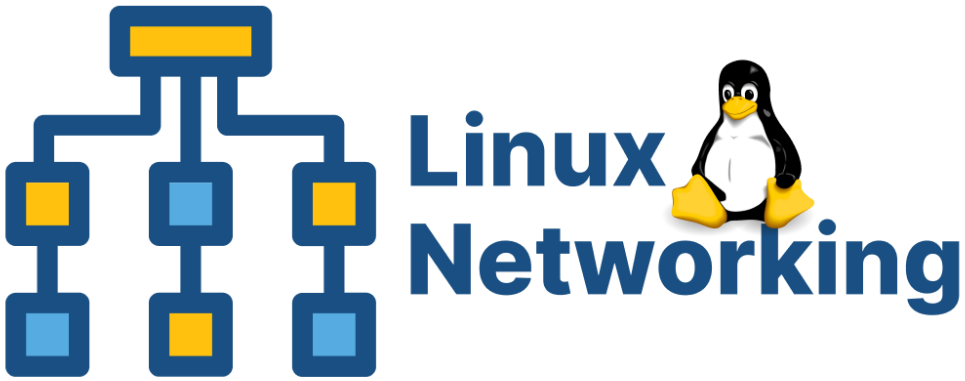
WEEK 8

Intro to Linux Networking & Application Runtime



Linux Networking Basics

1.1 IP Addresses and Interfaces

Every computer connected to a network has at least one IP address, a unique numerical label that identifies it on the network. Think of it as a postal address: without it, no one knows where to send data.

Linux systems have network interfaces, named connections to networks. Common interface names are eth0 (wired ethernet), wlan0 (Wi-Fi), and lo (loopback, the machine talking to itself). The loopback address 127.0.0.1 always refers to the local machine and is also accessible as localhost.

```
# View all network interfaces and their IP addresses
$ ip addr show

# Shorter, cleaner output
$ ip addr show | grep 'inet '

# Older alternative (still widely used)
$ ifconfig
```

In the output, look for a line like inet 192.168.1.5/24. The number before the / is your IP address. The /24 is the subnet mask, it defines which part of the address identifies the network vs the individual host.

1.2 Testing Connectivity with ping

ping sends small test packets to another machine and measures how long the reply takes. It is the first tool you reach for when diagnosing a network problem.

```
# Ping Google's DNS server (tests internet connectivity)
$ ping 8.8.8.8

# Ping by hostname (also tests DNS resolution)
$ ping google.com

# Send only 4 packets then stop
$ ping -c 4 google.com

# Ping the loopback, tests the local network stack
$ ping 127.0.0.1
```

If ping 8.8.8.8 works but ping google.com fails, the network is fine but DNS resolution is broken. This distinction immediately narrows down where the problem lies, a critical debugging skill.

1.3 DNS (Translating Names to IP Addresses)

DNS (Domain Name System) is the phone book of the internet. It translates human-readable names like github.com into machine-readable IP addresses like 140.82.121.4. Without DNS, you would have to memorise IP addresses for every website.

```
# Look up the IP address for a domain name
$ nslookup google.com
# More detailed DNS query
$ dig google.com

# Quick one-line answer
$ dig +short google.com
```

1.4 Checking Open Ports and Connections

A running application listens on a port, waiting for connections. Knowing which ports are open and which process owns them is essential for server management and debugging.

```
# Show all listening ports and the process using them
$ ss -tulnp

# Alternative using netstat (older systems)
$ netstat -tulnp

# Check if a specific port is in use
$ ss -tulnp | grep :5000

# Show all active network connections
$ ss -tp
```

Flag	Meaning
-t	Show TCP connections
-u	Show UDP connections
-l	Show only listening (waiting for connections) sockets
-n	Show numeric port numbers instead of service names
-p	Show the process name and PID using the socket

1.5 Tracing the Route to a Server

Traceroute shows every network hop (router) a packet passes through on its way to a destination. It helps diagnose where in the network a connection is slow or failing.

```
$ traceroute google.com

# If traceroute is not installed:
$ sudo apt install traceroute -y
```

Running Applications on Linux Servers

2.1 Binding an Application to a Port

When you run a server application, like your Flask app, it binds to a port and listens for incoming TCP connections on that port. The application stays running as long as the process is alive. In

production, applications are expected to keep running permanently, even after the terminal is closed or the developer logs out.

You have already done this with Flask (port 5000) and Python's HTTP server (port 8080). Now let us understand it more carefully.

```
# Start Flask on its default port 5000
$ cd ~/devops/week5 && python3 app.py

# In another terminal —> confirm it is listening
$ ss -tulnp | grep :5000

# Start Flask on a custom port
$ flask run --port=8000

# Bind to all interfaces (0.0.0.0) so other machines on the network can
connect
$ flask run --host=0.0.0.0 --port=5000
```

By default, Flask binds to 127.0.0.1 (localhost only). Using --host=0.0.0.0 makes it accessible from other machines. On a cloud server this is required so that users outside the machine can reach your app.

2.2 Running Applications in the Background

If you start Flask in a terminal and then close that terminal, the application stops. On a real server, applications must keep running indefinitely. There are several ways to achieve this.

Method 1: Using & (simple background job)

```
$ python3 app.py &
# [1] 4521   <- Job number and PID

$ jobs          # List background jobs
$ fg            # Bring back to foreground
$ kill %1       # Kill job number 1
```

Method 2: Using nohup (survives terminal close)

```
# nohup = 'no hang up' — keeps the process running after logout
# Output is redirected to nohup.out automatically
$ nohup python3 app.py &

# Check it is running
$ ps aux | grep app.py

# Redirect output to a specific log file
$ nohup python3 app.py > app.log 2>&1 &
```

The 2>&1 part redirects stderr (error output, stream 2) into the same place as stdout (normal output, stream 1). This ensures all output, both normal and error messages, goes into app.log.

Method 3: Using systemd (production standard)

systemd is the Linux service manager. It can start applications automatically at boot, restart them if they crash, and manage their logs. This is how real production services are managed.

```
# Create a service file for your Flask app
$ sudo nano /etc/systemd/system/flaskapp.service

[Unit]
Description=Samantha Flask App
After=network.target

[Service]
User=samantha
WorkingDirectory=/home/samantha/devops/week5
ExecStart=/usr/bin/python3 app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

```
# Reload systemd, enable and start the service
$ sudo systemctl daemon-reload
$ sudo systemctl enable flaskapp
$ sudo systemctl start flaskapp

# Check its status
$ sudo systemctl status flaskapp

# Stop and disable
$ sudo systemctl stop flaskapp
$ sudo systemctl disable flaskapp
```

Restart=always in the service file means systemd will automatically restart your app if it crashes. This is the minimum you need for any production service, it ensures your app recovers from unexpected errors without manual intervention.

Logs

3.1 Why Logs Matter in DevOps

Logs are the single most important tool for understanding what is happening on a running server. When an application crashes at 2am, when a user reports that a page is not loading, when a deployment breaks silently, logs are where you find the answer. A DevOps engineer who cannot read logs cannot debug production systems.

Every application writes log messages as it runs. These messages record events: a request was received, a database query ran, an error occurred, a user logged in. By reading the logs, you reconstruct exactly what happened and when.

3.2 System Logs

```
# Main system log —> everything the OS and services record
```

```
$ sudo cat /var/log/syslog

# Watch it live as new messages arrive
$ sudo tail -f /var/log/syslog

# Authentication log —> login attempts, sudo usage
$ sudo cat /var/log/auth.log

# Kernel messages —> hardware, drivers, boot
$ dmesg | tail -20

# systemd service logs (journalctl)
$ sudo journalctl -u flaskapp          # All logs for the flaskapp service
$ sudo journalctl -u flaskapp -f       # Follow live logs
$ sudo journalctl -u flaskapp --since '10 min ago'
```

3.3 Application Logs from Flask

Flask prints a log line to the terminal for every HTTP request it receives. When running with nohup or as a systemd service, these logs are written to a file. Here is what a Flask log line looks like and how to read it:

```
127.0.0.1 - - [05/Apr/2024 14:32:07] "GET /api/students HTTP/1.1" 200 -
127.0.0.1 - - [05/Apr/2024 14:32:09] "POST /api/students HTTP/1.1" 201 -
127.0.0.1 - - [05/Apr/2024 14:32:11] "GET /api/students/999 HTTP/1.1" 404 -
```

Part	Example	Meaning
Client IP	127.0.0.1	IP address of the machine that sent the request
Timestamp	[05/Apr/2024 14:32:07]	Date and time the request was received
Method + URL	GET /api/students	HTTP method and the endpoint that was called
HTTP Version	HTTP/1.1	Protocol version used
Status Code	200 / 404	Response code, was the request successful?

3.4 Searching and Filtering Logs with grep

Production logs can contain thousands of lines. grep filters them down to only the lines that matter.

```
# Show only error lines from syslog
$ sudo grep 'error' /var/log/syslog

# Case-insensitive search
$ sudo grep -i 'error' /var/log/syslog

# Show 3 lines before and after each match (context)
$ sudo grep -B 3 -A 3 'ERROR' app.log

# Count how many error lines exist
$ sudo grep -c 'error' /var/log/syslog
```

```
# Combine tail and grep — watch live errors only
$ sudo tail -f /var/log/syslog | grep -i 'error'
```

Networking Utilities Quick Reference

4.1 curl for Network Testing

You already used curl to test your Flask API in Week 5. It is also a powerful networking tool for checking if a server is reachable and measuring response times.

```
# Basic connectivity check —> is a server responding?
$ curl http://localhost:5000/api/health

# Show HTTP response headers
$ curl -I http://localhost:5000

# Measure total response time
$ curl -o /dev/null -s -w 'Total time: %{time_total}s\n' http://localhost:5000

# Test connectivity to an external server
$ curl -I https://github.com
```

4.2 wget (Downloading Files)

wget downloads files from the internet directly to your Linux machine from the terminal. It is useful for downloading configuration files, software packages, or test data onto a server.

```
# Download a file to the current directory
$ wget https://example.com/file.zip

# Download and save with a specific filename
$ wget -O myfile.zip https://example.com/file.zip

# Download quietly (no progress output)
$ wget -q https://example.com/file.txt
```

4.3 Full Command Reference

Command	Purpose	Key Example
ip addr show	View IP addresses and interfaces	ip addr show | grep inet
ping	Test connectivity to a host	ping -c 4 google.com
nslookup / dig	DNS lookup, name to IP	dig +short github.com
ss -tulnp	Show open ports and listening processes	ss -tulnp | grep :5000
traceroute	Trace the network path to a destination	traceroute google.com
curl	HTTP requests and connectivity testing	curl -I http://localhost:5000

wget	Download files from the internet	wget -O f.txt https://url
nohup ... &	Run a process that survives terminal close	nohup python3 app.py &
systemctl	Manage systemd services	systemctl status flaskapp
journalctl	View systemd service logs	journalctl -u flaskapp -f
tail -f	Watch a log file in real time	tail -f app.log
grep	Search and filter log output	grep -i error app.log

Practice Exercise

Part A: Network Investigation

1. Find your machine's IP address

```
$ ip addr show | grep 'inet '
```

2. Test connectivity and DNS

```
$ ping -c 4 8.8.8.8
$ ping -c 4 github.com
$ dig +short github.com
```

3. Check what ports are currently open on your machine

```
$ ss -tulnp
```

Part B: Running Flask in the Background

4. Navigate to your Week 5 Flask project

```
$ cd ~/devops/week5
```

5. Start Flask in the background using nohup and redirect output to a log file

```
$ nohup python3 app.py > flask_app.log 2>&1 &
$ echo "Flask PID: $!"
```

6. Confirm the process is running and the port is open

```
$ ps aux | grep app.py
$ ss -tulnp | grep :5000
```

7. Send a request and watch the log update

```
$ curl http://localhost:5000/api/students
$ cat flask_app.log
```

8. Watch the log live while sending more requests

```
# Terminal 1 —> watch the log
$ tail -f flask_app.log

# Terminal 2 —> send requests
$ curl http://localhost:5000/api/students
$ curl http://localhost:5000/api/students/1
$ curl http://localhost:5000/api/students/999
```

9. Stop the Flask process

```
$ kill $(ps aux | grep 'python3 app.py' | grep -v grep | awk '{print $2}')
```

Part C: systemd Service

10. Create and enable a systemd service for your Flask app (use the template from Section 2.2)

```
$ sudo nano /etc/systemd/system/flaskapp.service
# Fill in your username and correct path
$ sudo systemctl daemon-reload
$ sudo systemctl start flaskapp
$ sudo systemctl status flaskapp
```

11. Test that it survives, stop it, then check it restarts

```
$ sudo systemctl stop flaskapp
$ sudo systemctl start flaskapp
$ sudo journalctl -u flaskapp --since '2 min ago'
```

Lab Evaluation Tasks

Complete both tasks. The instructor will verify your running service, open ports, and log output directly in the terminal. Each task carries equal marks.

Lab Task 1: Deploy Flask as a Background Service

Deploy your Week 5 Flask CRUD API as a persistent background service using nohup. The app must be running and reachable when the instructor checks — even if the original terminal was closed.

1. Navigate to your Week 5 Flask project directory
2. Start the Flask app in the background using nohup, redirecting all output to ~/devops/week8/flask_production.log
3. Verify the process is running: ps aux | grep app.py — note the PID
4. Verify port 5000 is listening: ss -tulnp | grep :5000
5. Send three curl requests to your API: GET /api/students, POST a new student, GET /api/health
6. Show the instructor flask_production.log — it must contain log lines for all three requests, including timestamps and status codes
7. Use grep to filter only the lines with status code 404 from the log: grep '404' flask_production.log
8. Demonstrate stopping and restarting the service using kill and nohup